Lesson 6.3 — Pivots and Internal State

Lesson 6.3 — Pivots and Internal State

> **Some state belongs to a single component.** This lesson is about a chart that owns its own UI state — and about pivoting data so the *match* (not the *team*) becomes the unit of analysis.

By the end of this lesson you will:

- Understand the difference between **pivoting** and **summarizing** data.
- Build `<LeagueAgreementChart>` — a chart with **internal tab state** (`useState` inside the chart, not lifted up).
- Know precisely when to lift state vs keep it local.
- Render **conditional `<Bar>` elements** based on whether matches are finished.
- Convert the long, ugly transform inside `AnalyticsTab` into a clean, named helper function.

This is the case study that ties the entire chapter together. It also happens to be one of the most asked-about patterns in senior React interviews: *"When do you lift state?"*

---

1. What is a pivot?

A **pivot** is a transform where you change *what each row represents*.

- Before: `team -> picks` (each row is one team).
- After: `match -> backers` (each row is one match, plus how the league split on it).

The same underlying data, viewed through a different lens.

If you've ever used Excel pivot tables, you've already done this. In React, we do it with `.map()`, `.filter()`, and sometimes `.reduce()`.

Why pivot?

Because **the chart's x-axis dictates the data shape**.

- Want a chart with one bar per *team*? You need an array where each element is a team. That's the shape we used in Lesson 2.
- Want a chart with one bar per *match*? You need an array where each element is a match. That's a pivot.

The chart can't pivot for you. The data has to arrive in the shape the chart expects.

---

2. The question this chart answers

Look back at your standings. You can see who's winning. But you can't see **where the league agreed and where it disagreed**.

For each match in Round 1, ask:

- *How many of the 8 teams picked the actual winner?*
- *How many got it wrong?*

That tells you which matches were **upsets** (most teams got it wrong) and which were **chalk** (everyone agreed and was correct).

For unfinished matches, ask a different question:

- *Of the 8 teams, how many backed player 1? How many backed player 2?*

That tells you where consensus lies *before* the match has been played.

This is the chart we're building.

---

3. Plan the data shape first

Always plan the data shape before touching the chart code.

For each match, we need an object like this:

```
{
  match: 'Williams v Wakelin',  // x-axis label
```

```
  finished: true,
  correct: 5,
  wrong: 3,
  pending: 0,
}
```

Or, for an unfinished match:

```
{
  match: 'Murphy v Wu',
  finished: false,
  p1Name: 'Murphy',
  p2Name: 'Wu',
  p1Backers: 2,
  p2Backers: 6,
  pending: 8,
}
```

Two shapes — one for finished, one for pending — but with **enough overlap that the same chart can render both** (we just toggle which `<Bar>` components mount based on `finished`).

This is a pattern you'll see often: design *one* row shape that satisfies *all* render branches.

---

4. Define the type contract

Open `components/analytics/LeagueAgreementChart.tsx`. Top of the file:

```
'use client';

import { useState } from 'react';
import {
  BarChart,
  Bar,
  XAxis,
  YAxis,
```

```
  CartesianGrid,
  Tooltip,
  ResponsiveContainer,
  Legend,
} from 'recharts';

export interface AgreementDatum {
  match: string;
  finished: boolean;
  correct?: number;
  wrong?: number;
  pending: number;
  p1Name?: string;
  p2Name?: string;
  p1Backers?: number;
  p2Backers?: number;
}

export interface AgreementRound {
  id: string;
  label: string;
  subtitle: string;
  data: AgreementDatum[];
}
```

Why these types?

- **`AgreementDatum`** describes *one bar* on the chart.
- **`AgreementRound`** describes *one tab* — Round 1, Round 2, etc. — bundling the round's name, subtitle, and the array of bars.

Why are `correct`, `wrong`, `p1Name`, `p1Backers`, `p2Name`, `p2Backers` all optional?

Because each row uses *one set or the other*, never both:

- A finished row uses `correct` and `wrong`.

- A pending row uses `p1Backers` and `p2Backers`.

TypeScript would let us model this as a **discriminated union** (a stricter type), and that's what a senior engineer might do. We're using optional fields here because it keeps the data builder simple. **Both are valid — pick the one that fits your team's style.**

> **Interview answer:** "I'd use a discriminated union if the consumer of the type does a lot of branching on `finished`. I went with optional fields here because the chart is the only consumer, and the branching happens once."

Why `export interface`?

So that **other files can import `AgreementDatum`** and use it as the return type of helper functions. Concretely, `AnalyticsTab.tsx` will do:

```
import LeagueAgreementChart, { type AgreementDatum } from '../analytics/Leag
```

The `type` keyword in the import is a TypeScript signal: *"This is a type-only import — it disappears at compile time."* That keeps your runtime bundle clean.

---

5. The component shell

Add the props interface and the component skeleton:

```
interface LeagueAgreementChartProps {
  rounds: AgreementRound[];
}

export default function LeagueAgreementChart({ rounds }: LeagueAgreementChar
  const [active, setActive] = useState<string>('r1');
  const round = rounds.find((r) => r.id === active) || rounds[0];
  const isPendingRound = round.data.every((d) => !d.finished);

  return (
    <div /* ... container styles ... */>
      {/* tab buttons */}
      {/* chart */}
```

```
    </div>
  );
}
```

Read this carefully — there are three big ideas here.

5.1. `useState` is local. The `active` state — *which round tab is selected* — lives inside this component. The orchestrator at the top of the app **does not know it exists**.

That's deliberate.

> **Lifting state up** is a real React principle, but the rule is: **lift state only when something else needs it.** Nothing outside this chart cares which tab is open. So it stays here.

If, three months from now, the URL needs to reflect the active tab (so a user can share a link to "League Agreement Final"), we'd lift this into a route param. Until then: local.

```
const round = rounds.find((r) => r.id === active) || rounds[0];
```

5.2. The `find()` fallback. If `active` somehow points to a round that doesn't exist (typo, removed round), we fall back to the first round. **Defensive programming.** The user sees something instead of a crash.

```
const isPendingRound = round.data.every((d) => !d.finished);
```

5.3. Derived booleans live above the JSX. Compute once at the top of the render. Use the boolean inside JSX. Don't write `round.data.every((d) => !d.finished)` twice — that's a smell.

---

6. Build the tab buttons

Inside the container `<div>`:

```
<div style={{ display: 'flex', gap: 8, flexWrap: 'wrap', marginBottom: 20 }}>
  {rounds.map((r) => {
    const isActive = r.id === active;
    return (
      <button
        key={r.id}
        onClick={() => setActive(r.id)}
        style={{
          padding: '8px 16px',
          borderRadius: 8,
          border: isActive ? '2px solid #0F5132' : '2px solid #E5E7EB',
          background: isActive ? '#0F5132' : '#FFFFFF',
          color: isActive ? '#FBBF24' : '#374151',
          fontWeight: 700,
          fontSize: 13,
          cursor: 'pointer',
          fontFamily: 'inherit',
          letterSpacing: '0.5px',
          transition: 'all 0.15s',
        }}
      >
        {r.label}
      </button>
    );
  })}
</div>
```

Why a simple button row instead of a `<select>` or radio?

Because we have a small, fixed set of options (5 rounds) and we want each option to be a **prominent target**. Buttons in a row are touch-friendly, glanceable, and don't hide options behind a click.

Why one `onClick={() => setActive(r.id)}` per button?

Each button needs its own handler that knows its `id`. The arrow function creates a closure that captures `r.id`. This is standard React.

You may have read that creating arrow functions in render is "expensive" or "bad for performance." For a list of 5 buttons that re-renders only when `active` changes? **Don't worry about it.** Optimize when measurements say to, not before.

---

7. Build the chart

After the tab row, add:

```
<ResponsiveContainer width="100%" height={Math.max(280, round.data.length * 
  <BarChart data={round.data} margin={{ top: 10, right: 20, bottom: 80, left
    <CartesianGrid strokeDasharray="3 3" stroke="#FDE68A" />
    <XAxis
      dataKey="match"
      tick={{ fontSize: 11, fontWeight: 600, fill: '#1F2937' }}
      angle={-45}
      textAnchor="end"
      interval={0}
      height={90}
    />
    <YAxis tick={{ fontSize: 12, fill: '#1F2937' }} domain={[0, 8]} />
    <Tooltip contentStyle={{ background: '#FFFBEB', border: '2px solid #FBBF
    <Legend wrapperStyle={{ fontSize: 13, fontWeight: 600 }} />

    {isPendingRound && (
      <Bar dataKey="p1Backers" stackId="a" fill="#0F5132" name="Backed first
    )}
    {isPendingRound && (
      <Bar dataKey="p2Backers" stackId="a" fill="#B45309" name="Backed secon
    )}
    {!isPendingRound && (
      <Bar dataKey="correct" stackId="a" fill="#16A34A" name="Correct picks"
```

```
    )}
    {!isPendingRound && (
      <Bar dataKey="wrong" stackId="a" fill="#DC2626" name="Wrong picks" />
    )}
  </BarChart>
</ResponsiveContainer>
```

Three details worth circling.

```
height={Math.max(280, round.data.length * 30 + 120)}
```

7.1. Dynamic height. Some rounds have 16 matches (R1), some have 1 (Final). A fixed height of 280px would crush 16 bars together. We compute the height from the data length: 30px per row plus padding, with a floor of 280.

This is a tiny detail that **separates beginner React from production React.** Layouts that adapt to data, not the other way around.

```
{isPendingRound && <Bar dataKey="p1Backers" ... />}
{!isPendingRound && <Bar dataKey="correct" ... />}
```

7.2. Conditional `<Bar>` rendering. Recharts traverses its children to figure out which series to draw. If a `<Bar>` is not rendered, that series doesn't appear. So our toggle between *finished* and *pending* visualizations is just **conditional JSX**.

This is React composition at its best: you don't reach for an imperative API to "add a bar" or "remove a bar." You just describe which `<Bar>`s should exist for the current state.

7.3. `stackId="a"`. All bars in the same stack share an `id`. Recharts treats them as one stacked column. For a finished match, `correct` (green) is stacked on top of `wrong` (red), and the column total is always 8. For a pending match, `p1Backers` (green) is stacked on `p2Backers` (orange), and again the total is 8. **Visually consistent.**

---

8. Build the pivot helper in `AnalyticsTab.tsx`

Open `components/tabs/AnalyticsTab.tsx`. We need a helper that turns a list of matches into the chart-ready array.

```
const buildAgreementData = (
  matches: Match[],
  pickKey: 'r1' | 'r2' | 'qf' | 'sf' | 'final'
): AgreementDatum[] =>
  matches.map((m, i) => {
    const matchLabel = `${m.p1.split(' ').slice(-1)[0]} v ${m.p2.split(' ').
    if (m.winner) {
      const correct = teams.filter((t) => {
        const pick =
          pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null;
        return pick === m.winner;
      }).length;
      return {
        match: matchLabel,
        correct,
        wrong: 8 - correct,
        pending: 0,
        finished: true,
      };
    } else {
      const p1Backers = teams.filter((t) => {
        const pick =
          pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null;
        return pick === m.p1;
      }).length;
      const p2Backers = teams.filter((t) => {
        const pick =
          pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null;
        return pick === m.p2;
      }).length;
      return {
        match: matchLabel,
```

```
        p1Name: m.p1.split(' ').slice(-1)[0],
        p2Name: m.p2.split(' ').slice(-1)[0],
        p1Backers,
        p2Backers,
        pending: 8,
        finished: false,
      };
    }
  });
```

This is the messiest function in the entire project. Let's break it down piece by piece.

8.1. Why is this a function inside the component, not a `lib/` helper?

Two reasons:

1. **It needs `teams` from the props.** Putting it in `lib/` would force us to pass `teams` in every call and would gain us nothing because `teams` is already in scope here.
2. **It's only used here.** No other component will ever build agreement data. Premature extraction is a real cost.

   Interview answer: "I extract to `lib/` when (a) the function is pure and (b) more than one component needs it. This one fails (b) for now, so I keep it local."

8.2. What is `pickKey` doing?

`pickKey` is a **string parameter** that tells the function *which array on the team to read*: `t.r1`, `t.r2`, `t.qf`, `t.sf`, or — special-cased — `t.final`.

This is how we avoid copy-pasting the function five times (one per round). One function, parametrized by which slot on the team we read.

The TypeScript type `'r1' | 'r2' | 'qf' | 'sf' | 'final'` is a **string literal union**. It restricts the caller to exactly those five strings. If you pass `'foo'`, TypeScript errors.

8.3. Why the special case for `'final'`?

Because the team type has `t.final: string` (a single player name), not `t.final: string[]` (an array of picks). So:

- For rounds, the team's pick for match `i` is `t[pickKey][i]`.
- For the final, the pick is just `t.final` (no array index).

Hence the ternary:

```
pickKey === 'final' ? t.final : t[pickKey] ? t[pickKey][i] : null
```

Read this top to bottom:

1. Is `pickKey` literally `'final'`? Yes → use `t.final`.
2. Otherwise, does `t[pickKey]` exist? Yes → grab the `i`th element.
3. Otherwise → `null` (the team didn't pick anything for this round).

The middle check (`t[pickKey] ?`) is defensive — early in the season, some teams might not have R2 picks yet.

8.4. The label trick.

```
const matchLabel = `${m.p1.split(' ').slice(-1)[0]} v ${m.p2.split(' ').slice
```

`'Mark Williams'.split(' ')` → `['Mark', 'Williams']`. `.slice(-1)` → `['Williams']`. `[0]` → `'Williams'`.

Net effect: keep the **last word** (the surname) of each player's full name.

Why? Because chart x-axes have very little horizontal space. `'Mark Williams v Lei Peifan'` is a label that gets crushed into nothing. `'Williams v Peifan'` actually reads.

8.5. The vote count.

```
const correct = teams.filter((t) => /* pick equals winner */).length;
```

This is the **counting-by-filtering** idiom. Count how many array elements satisfy a predicate.

Could you use `.reduce((acc, t) => pick === winner ? acc + 1 : acc, 0)`? Yes. But `.filter().length` reads better here.

> **Interview answer:** "I use `.filter().length` when readability beats one extra pass over the array. For arrays of millions of elements, `.reduce()` is faster because it avoids the intermediate array. For 8 teams? It doesn't matter."

8.6. The implicit invariant.

We have **8 fantasy teams**. Every team picks for every match. So `correct + wrong = 8`. Always.

That's why we can write:

```
return { correct, wrong: 8 - correct, ... };
```

When the **invariant is structural** (it can't change without changing the whole app), it's safe to hardcode the 8. If you wanted to be extra-defensive, you could write `wrong: teams.length - correct` — both are fine. The hardcoded 8 is a tradeoff for clarity in a UI label that says *"out of 8 teams"*.

---

9. Wire it up

Below the helper definition (still inside `AnalyticsTab`), build one array per round:

```
const r1Universal = buildAgreementData(ROUND1_MATCHES, 'r1');
const r2Universal = buildAgreementData(ROUND2_MATCHES, 'r2');
const qfUniversal = buildAgreementData(QF_MATCHES, 'qf');
const sfUniversal = buildAgreementData(SF_MATCHES, 'sf');
const finalUniversal = buildAgreementData(FINAL_MATCH, 'final');
```

Then render the chart, passing all five rounds at once:

```
<LeagueAgreementChart
  rounds={[
    { id: 'r1', label: 'Round 1', subtitle: 'Last 32 — Best of 19', data: r1
    { id: 'r2', label: 'Round 2', subtitle: 'Last 16 — Best of 25', data: r2
    { id: 'qf', label: 'Quarter-Finals', subtitle: 'Last 8 — Best of 25', da
    { id: 'sf', label: 'Semi-Finals', subtitle: 'Last 4 — Best of 33', data:
    { id: 'final', label: 'Final', subtitle: 'Murphy v Wu — Best of 35', dat
  ]}
/>
```

Click around the tabs. The chart's internal `useState` flips `active`, and the right round renders.

---

10. The big question: lift state, or keep it local?

This is the question senior interviewers love.

When to keep state local

- Only this component reads it.
- Only this component writes it.
- The state doesn't need to survive remounting.
- The state doesn't need to be shared with the URL or persistence.

The chart's `active` tab passes all four tests. **Local.**

When to lift state up

- A *sibling* component needs to read it.
- A *parent* component needs to react to it.
- The state should persist (e.g., URL param, localStorage).
- Two pieces of state must stay in sync.

For example, `selectedTeam` in our orchestrator — that one *had* to be lifted, because both the standings tab (which sets it via clicking) and the predictions tab (which reads it via the team selector) need to share the same value.

The diff is roughly:

Concern	Local state	Lifted state
Component count needing it	1	2+
Persistence required	No	Sometimes
URL reflects it	No	Sometimes
Resets on remount	OK	Often not OK
Test isolation	Easier	Harder

> **Interview-grade answer:** "I default to local. I only lift state when the absence of lifting causes a bug — typically when two components need to be in sync,

or when state needs to survive a route change. Lifting state too eagerly turns every parent into a state monolith, and that's the start of every 'too much prop drilling' complaint I've ever heard."

---

11. Common mistakes I want you to avoid

11.1. Building the data inside the chart

This is wrong:

```
function LeagueAgreementChart({ teams, matches }: Props) {
  const data = matches.map(m => /* pivot logic */);
  // ...
}
```

The chart now knows *too much*. It knows what a `Team` is, what a `Match` is, how scoring works. Reuse just died.

The right shape: the chart accepts **already-pivoted data**. The orchestrator does the pivot. Loose coupling.

11.2. Storing the pivoted data in `useState`

This is also wrong:

```
const [data, setData] = useState(buildAgreementData(...));
```

Why? Because `buildAgreementData` is **derived** from `teams`. If `teams` changes, `data` is stale. `useState` is for state that the *user* mutates. **Derived data does not belong in state.**

The right approach: just compute it on every render. (Or wrap in `useMemo` if profiling shows it's slow.)

11.3. Lifting `active` into the orchestrator "just in case"

A team I worked with did this once. Every chart's "active tab" lived in a top-level reducer. Why? *"In case we want to share active tabs across charts."*

We never did. The reducer ballooned. Every chart re-rendered when *any* tab changed. We eventually pulled it all back down to the chart level.

Don't optimize for hypothetical future requirements. Optimize for now. Refactor when "now" actually demands it.

---

12. Vibe prompt you would have used

> "I have an array of fantasy teams, each with picks for 5 rounds, and I have arrays of matches per round. I want a Recharts bar chart where each bar is one match, stacked: green = how many teams picked the winner, red = how many got it wrong. For unfinished matches, swap to green = backed player 1, orange = backed player 2. Use a button row to switch rounds, and put the active-round state inside the chart component (don't lift it). Type everything strictly. Show me how to pivot the team-data into match-data with a single helper."

Notice the structure:

1. **Inputs** described precisely (teams shape, matches shape).
2. **Output** described visually (stacked bar, color rules).
3. **State location specified** (inside the chart, don't lift).
4. **Type discipline asserted** (strict typing).
5. **One specific request** (pivot helper).

This is the kind of prompt that produces high-quality code. The *vague* version — *"make a chart of who agreed with each match"* — produces a mess.

---

13. CHECK YOURSELF

Don't proceed until you can explain each of these:

- ☐ What does **pivot** mean? Give an example.
- ☐ Why is `useState` for the active tab kept inside the chart and not in the orchestrator?
- ☐ What is a **string literal union** type? Why use it for `pickKey`?
- ☐ What's the difference between **derived data** and **state**? Where does each belong?

- ☐ Why are `correct`, `wrong`, `p1Backers`, `p2Backers` optional fields on `AgreementDatum`?
- ☐ What would change if you used a **discriminated union** instead?
- ☐ Why use **conditional `<Bar>` rendering** instead of one `<Bar>` whose `dataKey` is computed?
- ☐ Why is the chart's height computed from `round.data.length`?
- ☐ What's the rule for *when to lift state up*?
- ☐ Why don't we put `buildAgreementData` in `lib/`?

If any of these is fuzzy, scroll back. The next chapter (Mastery + Interview Prep) assumes all of them are solid.

---

14. Where you are now

You've finished Chapter 6. Take stock:

- **Chapter 1** taught you the problem and the data model.
- **Chapter 2** scaffolded the project and typed the data.
- **Chapter 3** built your first component.
- **Chapter 4** taught you state, hooks, and the orchestrator pattern.
- **Chapter 5** taught you composition and component extraction.
- **Chapter 6** taught you data visualization, transforms, and where state lives.

You can now *build* the app. Chapter 7 will teach you how to *talk about it* — testing, deployment, and what to say in an interview when someone asks "walk me through your project."

What's next

Open `chapter-07-mastery-and-interviews/README.md`.

It's the closing chapter. We tighten everything, add the safety nets you haven't built yet (loading states, accessibility, error handling), and rehearse the interview narrative.